Development, Testing, Deployment and Monitoring

Project: DevOps Lifecycle and Automated Web Server Configuration Using Ansible

Submitted by:

Aakanksha Sah 202301070148 CNMD Lab batch-03

1. Project Overview

This project focuses on understanding the DevOps lifecycle through the automation of web server configuration using Ansible. The selected application scenario involves configuring an Nginx web server and deploying a static HTML webpage.

Ansible is an open-source automation tool used for configuration management, application deployment, and infrastructure automation. It uses inventory files to identify target hosts and playbooks written in YAML to define the tasks to be performed.

In this project, the Ansible playbook is designed to install Nginx, ensure that the service is enabled and running, and deploy a custom HTML homepage to the web server's document root.

The project demonstrates how automation can reduce manual configuration effort, improve consistency, and make deployment procedures repeatable.

Main project files:

- inventory.ini — Defines the target host.
- setup_webserver.yml — Contains the Ansible automation tasks.
- index.html — Contains the webpage to be deployed.
- README.md — Provides project details and execution instructions.

2. Development Stage

The development stage involves identifying the application requirements, preparing the configuration files, and defining the desired state of the web server.

2.1 Development Activities

The following activities are included in this stage:

1. Requirement identification: The objective is to automate the installation and configuration of an Nginx web server.
2. Inventory preparation: An Ansible inventory file is created to specify the target machine. In this project, the example configuration uses localhost with a local connection.

3. Playbook development: An Ansible playbook is written to install Nginx, start the service, enable it at system startup, and deploy the HTML file.
4. Webpage development: A simple HTML webpage is created to display a message confirming the intended web server configuration.
5. Project organization: The inventory, playbook, HTML file, and instructions are stored in a single project directory.
6. Version control: Git and GitHub can be used to maintain project files and track changes.

2.2 Development Output

The expected output of this stage is a set of configuration files that define how the web server should be configured.

The inventory identifies the target, the playbook describes the automation process, and the HTML file provides the content to be deployed.

3. Testing Stage

The testing stage verifies whether the inventory and playbook are correctly structured and whether the desired web server configuration can be achieved.

Testing is important because errors in YAML syntax, inventory configuration, permissions, or service settings may prevent successful deployment.

3.1 Testing Activities

The following tests are planned for this project.

Test	Command or Procedure	Expected Result
Ansible installation check	ansible --version	Displays the installed Ansible version.
Inventory validation	ansible-inventory -i inventory.ini --list	Displays the configured target host.
Playbook syntax check	ansible-playbook -i inventory.ini setup_webserver.yml --syntax-check	Confirms that the playbook has valid syntax.
Playbook execution	ansible-playbook -i inventory.ini setup_webserver.yml	Executes the defined configuration tasks.
Service verification	systemctl status nginx	Shows whether Nginx is active and running.

Webpage verification	Open http://localhost in a browser	Displays the deployed homepage if configuration succeeds.

Current testing status: These tests have not yet been executed in the project environment. The results listed above are expected outcomes, not verified execution results.

3.2 Testing Approach

The project follows these testing approaches:

- Syntax testing: Checks the YAML structure and task definitions of the playbook.
- Inventory testing: Verifies that Ansible can identify the configured target host.
- Functional testing: Checks whether Nginx is installed and running after playbook execution.
- Content testing: Verifies whether the expected HTML page is served by Nginx.
- Repeatability testing: Checks whether running the playbook again preserves the desired configuration without making unnecessary changes.

These tests help identify configuration errors and provide confidence in the automation process.

4. Deployment Stage

The deployment stage applies the configuration defined in the Ansible playbook to the target machine.

For this project, deployment is intended to automate the installation and configuration of Nginx and publish the sample HTML webpage.

4.1 Deployment Procedure

The planned deployment process consists of the following steps:

1. Prepare a supported Linux environment with Python and Ansible installed.
2. Place inventory.ini, setup_webserver.yml, and index.html in the same project directory.
3. Validate the inventory and check the playbook syntax.
4. Execute the playbook using the appropriate administrative privileges.
5. Review the Ansible output and identify any failed tasks.
6. Verify that the Nginx service is active.
7. Open http://localhost in a browser to check whether the custom webpage is displayed.

4.2 Deployment Configuration

The Ansible playbook is designed to perform the following operations:

- Install Nginx using the APT package manager.
- Ensure that the Nginx service is started.
- Enable Nginx to start automatically when the system boots.
- Copy index.html to /var/www/html/index.html.

These tasks are suitable for a typical Debian- or Ubuntu-based system. The configuration may need modification for other Linux distributions.

The playbook uses Ansible's privilege-escalation feature to perform administrative tasks.

4.3 Deployment Benefits

Automated deployment offers several advantages:

- Reduces repetitive manual configuration.
- Improves consistency between deployments.
- Makes the setup process easier to repeat.
- Simplifies maintenance and future modifications.
- Reduces the likelihood of human error.

4.4 Error Handling and Recovery

If deployment encounters an error, the administrator should examine the failed task and its error message, correct the configuration, and rerun the relevant validation checks.

In a production environment, configuration changes should be tested on a non-production machine before being applied to a live server. Backups of important files should also be maintained to support recovery.

5. Monitoring Stage

The monitoring stage checks whether the deployed service remains available and operates as expected.

For this basic project, monitoring focuses on the Nginx service status, HTTP response, webpage content, and service logs.

5.1 Monitoring Activities

Monitoring Item	Command or Procedure	Expected Healthy State
Service availability	systemctl is-active nginx	Returns active.

HTTP response	curl -I http://localhost	Returns a successful HTTP response, typically 200 OK.
Webpage content	Open http://localhost in a browser	Displays the expected homepage.
Service logs	journalctl -u nginx --since "10 minutes ago"	No unexpected errors affecting service operation.
Nginx logs	Inspect /var/log/nginx/access.log and /var/log/nginx/error.log	Requests are logged and errors can be investigated.

Note: These commands and results describe the planned monitoring procedure. They have not been verified in the current environment.

5.2 Monitoring and Troubleshooting

The following actions can be taken if a problem is detected:

1. If Nginx is inactive, inspect the service status and logs to identify the cause.
2. If the webpage cannot be accessed, verify that Nginx is running and that the HTML file exists at the configured document root.
3. If an HTTP error occurs, inspect the Nginx error logs and review the server configuration.
4. Record the issue, diagnostic findings, and corrective action for future reference.

For larger applications, monitoring can be extended using centralized logging, automated health checks, performance metrics, alerting systems, and continuous integration/continuous deployment (CI/CD) tools.

6. DevOps Workflow Summary

The four main stages in this project work together as follows:

Stage	Main Activity	Expected Output
Development	Prepare inventory, playbook, and webpage.	Configuration and application files.
Testing	Validate syntax and verify functionality.	Test results and identified errors.
Deployment	Execute the playbook to configure Nginx.	Configured web server and deployed webpage.
Monitoring	Check service availability and inspect logs.	Service-health information and troubleshooting data.

The workflow supports the DevOps principle of automating repetitive tasks and continuously validating the application environment.

7. Implementation Status and Evidence

The project documentation, inventory file, Ansible playbook, and sample HTML page have been prepared. However, the practical implementation has not yet been executed because the local WSL/Ubuntu environment could not be initialized successfully.

As a result, successful playbook execution, Nginx service status, and webpage accessibility have not been verified.

To complete the practical implementation, the project should be executed in a supported Linux environment. The following evidence should then be collected:

1. Screenshot showing the Ansible version.
2. Screenshot showing successful inventory validation.
3. Screenshot showing playbook syntax validation and execution output.
4. Screenshot showing the Nginx service status.
5. Screenshot showing the deployed webpage in a browser.
6. A recorded demonstration explaining the workflow and showing the actual execution.

All screenshots should be captured from real execution. Until these checks are performed, the results should be treated as planned outcomes rather than confirmed results.

8. Conclusion

This project demonstrates how Ansible can be used to automate web server configuration as part of the DevOps lifecycle.

The development stage defines the desired configuration, testing checks its correctness, deployment applies the configuration, and monitoring verifies service health. Using an Ansible playbook makes the process repeatable and reduces the need for manual configuration.

The prepared project files and testing plan provide a foundation for practical implementation. Actual execution and verification are still required to confirm that the web server is configured successfully.